

Overview

The first Homework set for [ECE 417 - Image Processing](#)

Signature

Alexandre Dufresne-Nappert

20948586

Note: I Added plenty of extra content to explain the logic and so that I can refer back to the material in the future. Sorry for makin this assignment so long

Question 1

Let $f[n, m]$ be a two-dimensional discrete signal defined on $\mathcal{Z}^2$. Consider the signal :

$$f[n, m] = \cos(2\pi(\alpha \cdot n + \beta \cdot m))$$

where α and β are real constants.

Question 1.a

Derive the two-dimensional discrete-space Fourier transform (2-D DSFT) $F(\omega_1, \omega_2)$ of $f[n, m]$. Describe the locations and structure of the spectral components in the (ω_1, ω_2) plane.

Question 1.a Answer

First, we can note that the 2D Discrete Space Fourier Transform has the following formula :

$$F(\omega_1, \omega_2) = \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} f[n, m] e^{-i(\omega_1 \cdot n + \omega_2 \cdot m)}$$

Now, since we want to derive the Discrete Space Fourier Transform of the cosine function, we need to essentially input it into the function and compute it.

Unfortunately though, the cosine function will not play nice with the exponential form of the Fourier Transform. Therefor we must change the cosine to it's exponential form.

The general conversion of the form is as follows :

$$\cos(\theta) \rightarrow \frac{1}{2}(e^{i\theta} + e^{-i\theta})$$

And since $\theta = 2\pi(\alpha \cdot n + \beta \cdot m)$, we can directly plug it in as follows :

$$f[n, m] = \cos(2\pi(\alpha \cdot n + \beta \cdot m)) \rightarrow \frac{1}{2}(e^{i2\pi(\alpha \cdot n + \beta \cdot m)} + e^{-i2\pi(\alpha \cdot n + \beta \cdot m)})$$

Before moving on further, we need to establish the derivation for the Delta Property, which indicates that for a signal with a constant frequency, we can treat it as a Delta Dirac function as follows :

$$\sum_{n, m} (1) e^{-i(\omega_1 n + \omega_2 m)} = \underbrace{(2\pi)^2}_{\text{Range}=[-\pi, \pi]^2} \sum_{k_1, k_2} \delta(\omega_1 - 2\pi k_1, \omega_2 - 2\pi k_2)$$

Now, since the Discrete Space Fourier Transform follows Linear Properties, this means we can apply the Fourier Transform individually to each exponential term as follows :

$$\begin{aligned} F(\omega_1, \omega_2) &= \mathcal{F}(f[n, m]) \\ &= \mathcal{F}\left(\left(\frac{1}{2}(e^{i2\pi(\alpha \cdot n + \beta \cdot m)} + e^{-i2\pi(\alpha \cdot n + \beta \cdot m)})\right)\right) \\ &= \frac{1}{2}\mathcal{F}(e^{i2\pi(\alpha \cdot n + \beta \cdot m)}) + \frac{1}{2}\mathcal{F}(e^{-i2\pi(\alpha \cdot n + \beta \cdot m)}) \\ &= \frac{1}{2}(\mathcal{F}(e^{i2\pi(\alpha \cdot n + \beta \cdot m)}) + \mathcal{F}(e^{-i2\pi(\alpha \cdot n + \beta \cdot m)})) \end{aligned}$$

And now applying the Fourier Transform we get :

$$\begin{aligned} F(\omega_1, \omega_2) &= \frac{1}{2}\left(\sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{i2\pi(\alpha \cdot n + \beta \cdot m)} e^{-i(\omega_1 n + \omega_2 m)} + \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{-i2\pi(\alpha \cdot n + \beta \cdot m)} e^{-i(\omega_1 n + \omega_2 m)}\right) \\ &= \frac{1}{2} \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{i2\pi(\alpha \cdot n + \beta \cdot m)} e^{-i(\omega_1 n + \omega_2 m)} + e^{-i2\pi(\alpha \cdot n + \beta \cdot m)} e^{-i(\omega_1 n + \omega_2 m)} \\ &= \frac{1}{2} \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{i(2\pi\alpha \cdot n + 2\pi\beta \cdot m) - i(\omega_1 \cdot n + \omega_2 \cdot m)} + e^{-i(2\pi\alpha \cdot n + 2\pi\beta \cdot m) - i(\omega_1 \cdot n + \omega_2 \cdot m)} \\ &= \frac{1}{2} \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{-i((\omega_1 - 2\pi\alpha)n + (\omega_2 - 2\pi\beta)m)} + e^{-i((\omega_1 + 2\pi\alpha)n + (\omega_2 + 2\pi\beta)m)} \end{aligned}$$

And now applying the Delta Dirac Property, we get the following :

$$\begin{aligned}
F(\omega_1, \omega_2) &= \frac{1}{2} \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} e^{-i((\omega_1-2\pi\alpha)n+(\omega_2-2\pi\beta)m)} + e^{-i((\omega_1+2\pi\alpha)n+(\omega_2+2\pi\beta)m)} \\
&\rightarrow \frac{1}{2} ((2\pi)^2 \sum_{k_1, k_2} (\delta((\omega_1 - 2\pi\alpha) - 2\pi k_1, (\omega_2 - 2\pi\beta) - 2\pi k_2) + \delta((\omega_1 + 2\pi\alpha) - 2\pi k_1, (\omega_2 + 2\pi\beta) - 2\pi k_2))) \\
&= \frac{(2\pi)^2}{2} \sum_{k_1, k_2} (\delta(\omega_1 - 2\pi\alpha - 2\pi k_1, \omega_2 - 2\pi\beta - 2\pi k_2) + \delta(\omega_1 + 2\pi\alpha - 2\pi k_1, \omega_2 + 2\pi\beta - 2\pi k_2)) \\
&= \boxed{2\pi^2 \sum_{k_1, k_2} (\delta(\omega_1 - 2\pi\alpha - 2\pi k_1, \omega_2 - 2\pi\beta - 2\pi k_2) + \delta(\omega_1 + 2\pi\alpha - 2\pi k_1, \omega_2 + 2\pi\beta - 2\pi k_2))}
\end{aligned}$$

Now based off this answer, and the new shape of our Delta Dirac function, we would observe the Spectral Locations appear at the locations $\pm(2\pi\alpha, 2\pi\beta)$. Because that's when the Delta Dirac would be activated.

Question 1.b

Suppose we observe only a finite $N \times M$ patch of this signal and compute its 2-D discrete Fourier transform (DFT)

a) Under what conditions on α and β will the spectrum consist of two isolated DFT impulses (i.e., no spectral leakage)? b) If these conditions are not satisfied, describe qualitatively how the spectrum will change.

Question 1.b Answer

Now since we are using the Simpler Discrete Fourier Transform over a Patch Size of $N \times M$ then we can simply evaluate it.

We know that when we evaluate the Discrete Fourier Transform, the individual Frequencies become $\omega_1 = \frac{2\pi k}{N}$ where $k = 0 \rightarrow N - 1$ and $\omega_2 = \frac{2\pi l}{M}$ where $l = 0 \rightarrow M - 1$.

Since we know the frequency for the Transform is :

$$\omega_{01} = 2\pi\alpha$$

$$\omega_{02} = 2\pi\beta$$

Then we can find the condition for no leakage when the following must be true :

$$\omega_{01} = \frac{2\pi k_0}{N} \rightarrow 2\pi\alpha = \frac{2\pi k_0}{N} \rightarrow \alpha = \frac{k_0}{N}$$

$$\omega_{02} = \frac{2\pi l_0}{M} \rightarrow 2\pi\beta = \frac{2\pi l_0}{M} \rightarrow \beta = \frac{l_0}{M}$$

Therefor, we need to find the values k_0 and l_0 such that they are equal to α and β and no other conditions are true. This would mean that all other indexes compute to 0 in the frequency domain, while at the specific index spikes indicating the exact signals frequency.

Now if the condition is not satisfied, this would mean there is spectral leakage into other frequency bins of the Fourier Transform. So instead of a single frequency being identified for the Fourier Transform, the real frequencies magnitude would be reduced, and neighboring frequencies would have magnitudes slightly higher than 0.

If we were to take a very fine Fourier Transform, this would result in observing a Gaussian shape around the exact frequency.

Question 1.c

In image processing, oriented textures (for example fabric patterns or fingerprints) often resemble sums of terms of the form above.

Explain how the orientation of the pattern in the spatial domain is reflected in the geometry of the spectrum in the two-dimensional frequency plane.

Based on your answer, describe how the Fourier domain can be used to detect dominant orientations in an image.

Question 1.c Answer

Textures seen in images and movies such as fabric and fingerprints can be represented by oscillations in a signal that follow a specific direction. Since we know from 1.a, that a single cosine translates to 2 symmetric single frequency impulses, a sum of cosines would translate numerous symmetric frequency impulses. By using this we can understand the orientation of a pattern effectively.

Some of the properties we can observe from the frequency dimensions would be as follows :

- Perpendicularity : The lines that connect our spectral peaks $\pm(2\pi\alpha, 2\pi\beta)$ would be perpendicular to the lines / grid indexes in the spacial dimension, so depending on the values of α and β we can predict how perpendicular the pattern is compared to one of the axes (Example : if $\alpha = 0$ we know the patterns will be vertical since the frequencies would appear as vertical lines)
- Distance and Scale : We could understand the spacing between each ridge in the signal because the distance between each frequency can be computed using $\sqrt{(2\pi\alpha)^2 + (2\pi\beta)^2}$. If we have peaks that are far from the center of the grid, then we would have high frequency fine textures, where as if they are close to the center, we would see coarse low frequency textures.
- Orientation : We can also observe the direction in which the texture repeats itself by using the following angular mapping equation $\theta = \arctan(\beta/\alpha)$. This indicates which direction the textures repeating texture would flow.

On top of observing properties about a repetitive signal such as a texture, we can use the Fourier Transform to filter images and videos and remove certain components like watermarks. It can also be used to detect dominant orientations by searching for the maximum magnitude peaks in the DFT grid $|F[k, l]|$. By identifying the indices (k, l) where the energy is most concentrated, we can programmatically determine the exact angle of the texture ridges. Additionally, we can use the Convolution Theorem to apply directional filters in the frequency domain, which allows us to isolate specific patterns or remove periodic noise much more efficiently than we could in the spatial domain.

Question 2

Let $f[n, m]$ be an $N \times M$ image and consider the two-dimensional convolution

$$g[n, m] = \sum_k \sum_l h[k, l] f[n - k, m - l].$$

This operation can be viewed as a linear operator $g = \mathbf{H}f$

Question 2.a

Assume the filter is separable, $h[k, l] = h_1[k]h_2[l]$. Let H_1 and H_2 denote the one-dimensional convolution operators associated with $h_1[k]$ and $h_2[l]$, acting along the columns and rows of the image, respectively.

a) Show that the filtering operation can be written in matrix form as

$$g = H_1 f H_2^T$$

b) Explain why this representation implies that the filter kernel $h[k, l]$ must be a rank-one matrix.

c) Based on this observation, describe a practical test for determining whether a given finite filter is separable.

Question 2.a Answer

First, to show that $g = H_1 F H_2^T$, we can start by using the definition of a 2D Convolution with a separable kernel :

$$g[n, m] = \sum_k \sum_l h_1[k] h_2[l] f[n - k, m - l]$$

Now we can rearrange the sums because the terms $h_1[k]$ and $h_2[l]$ are can be separated into independent terms, this yields us the following :

$$g[n, m] = \sum_k h_1[k] \left(\sum_l h_2[l] f[n - k, m - l] \right)$$

Now we can say that the first Sum represents a 1D convolution of each row f with the h_2 kernel, and this row form convolution is equivalent to a transpose operation of $F H_2^T$. Then for the outer sum it is being applied on the columns, which is equivalent to a regular column convolution which can be applied to the entire result :

$$g[n, m] = \underbrace{\sum_k h_1[k]}_{H_1} \underbrace{\left(\sum_l h_2[l] f[n - k, m - l] \right)}_{F H_2^T}$$

Now if we combine the entire thing we get our expected result of :

$$g = H_1 F H_2^T$$

Now to explain, the definition of a separable kernel is when $h[k, l] = h_1[k]h_2[l]$, therefor if we treat the kernel h as a matrix H_{ker} each element ends up being the product of an element from a column vector v and a row vector u^T :

$$H_{ker} = \begin{bmatrix} h_1[1] \\ h_1[2] \\ \vdots \end{bmatrix} \begin{bmatrix} h_2[1] & h_2[2] & \dots \end{bmatrix} = \mathbf{v} \mathbf{u}^T$$

Finally, to determine if the filter H is seperable, we can use Singular Value Decomposition which uses the following steps :

1. We perform Single Value Decomposition on the kernel which yields :

$$H = U \Sigma V^T$$

- 2. We then look at the single values of σ_i on the diagonal of Σ .
- 3. The we test if the matrix has a single non-zero singular value, if so then the filter is separable.
- 4. In the case where it is separable, the 1D kernels can be extracted from the first column of U and the first row of V^T multiplied by $\sqrt{\sigma_1}$

Question 2.b

Consider the following 3×3 filters:

$$h_1 = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

$$h_2 = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

a)Determine whether each filter is separable.

b) If separable, find the corresponding one-dimensional filters.

c)Estimate the number of multiplications required to filter an $N \times M$ image using :

- direct two-dimensional convolution
- separable implementation

Question 2.b Answer

First, we can establish that a matrix is rank-one if all of its rows (or columns) are scalar multiples of each other.

2.b) a-b) Analysis of h_1 and h_2 Since both filters h_1 and h_2 are identical:

$$h_1 = h_2 = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

We need to start by looking at the rows of the matrix. We observed that $Row_2 = 2 \times Row_1$ and $Row_3 = 1 \times Row_1$. Since these rows are linearly dependent, on the first row, the rank of the matrix can be determined as 1.

Now we can find the filters, we need to find the vectors $\vec{v}$ and $\vec{u}$ such that $h = \vec{v}\vec{u}^T$. If we inspect the first columns we observe that :

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}, \quad \mathbf{u}^T = \begin{pmatrix} 1 & 2 & 1 \end{pmatrix}$$

And to verify, we can multiply things out to receive the full filter :

$$\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1(1) & 1(2) & 1(1) \\ 2(1) & 2(2) & 2(1) \\ 1(1) & 1(2) & 1(1) \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Therefor the 1D filters are $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$.

c) Multiplication Count Estimation For an image of size $N \times M$

Now for the multiplication count estimate. We start by using a 2D Convolution in a

Now for the multiplication count estimate. We start by using a Direct 2D Convolution. In this case, for every single pixel in the output image, we must perform a $K \times K$ sum of products (where K is the filter dimension).

We start by K^2 multiplication per pixel, which results in a total multiplication of $N \cdot M \cdot K^2$.

Now for our 3×3 filter, this is where $K = 3$ and which results in :

$$N \cdot M \cdot 3^2 = 9NM$$

Now if we look at the separable form, we can decompose it into two 1D filters which we can apply in sequence. Therefor it applies a $1 \times K$ filter across the rows, and then a $K \times 1$ filter across the columns.

Counting the operations this results in :

$$\underbrace{N \cdot M \cdot K}_{\text{Horizontal}} + \underbrace{N \cdot M \cdot K}_{\text{Vertical}} = 2NMK$$

And applying this to our 3×3 kernel, we get the following number of operations :

$$2NMK = 2NM(3) = 6NM$$

Therefor the separable form is the better way to go since it results in less multiplications since it has $3NM$ less than the direct multiplications.

Question 2.c

Suppose a filter kernel $h[k, l]$ is not separable, but its matrix representation h has singular value decomposition :

$$h = \sum_{i=1}^r \sigma_i u_i v_i^T$$

- a) Show how this decomposition allows the filter to be implemented as a sum of separable filters
- b) Explain how a rank- r approximation can be used to accelerate filtering.

Question 2.c Answer

a) Implementation as a Sum of Separable Filters : First, we can look at the Singular Value Decomposition (SVD) formula provided. The SVD breaks down a non-separable matrix into a sum of r terms, where each term is the product of a singular value σ_i , a column vector u_i , and a row vector v_i^T .

From the previous derivation in Question 2.a, we know that a filter is seperable if it can be written in the form of a outer product of 2 vectors. This is exactly what each value in Single Value Decomposition represent.

If we define each individual component as $h_i = \sigma_i u_i v_i^T$, then h is a rank one matrix and will have a separable filter.

Since convolution is a linear operation, we can distribute the operations across sums as follows :

$$\begin{aligned} g[n, m] &= f[n, m] * h[k, l] \\ &= f[n, m] * \left(\sum_{i=1}^r \sigma_i u_i v_i^T \right) \\ &= \sum_{i=1}^r \left(f[n, m] * (\sigma_i u_i v_i^T) \right) \end{aligned}$$

This shows that the total output g is simply the sum of r independent separable convolutions. Practically, this means we can filter the image with u_i and v_i^T for each i , scale by σ_i , and add them all together at the end.

b) Rank-r Approximation for Acceleration : In many real-world filters, the singular values σ_i are sorted such that $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$. Often, the first few singular values contain the vast majority of the "energy" or information of the filter, while the rest are very small.

We can accelerate the filtering process by performing a low-rank approximation. Instead of summing all r terms, we only sum the first k terms (where $k < r$):

$$\hat{h} = \sum_{i=1}^k \sigma_i u_i v_i^T$$

The computational cost for a direct 2D convolution is $O(K^2)$ per pixel. For the separable SVD approach, the cost is $O(k \cdot 2K)$ per pixel.

As long as $2k < K$, we significantly reduce the number of multiplications required while maintaining a result that is mathematically very close to the original non-separable filter. This allows us to process high-resolution images much faster with only a negligible loss in filtering quality.

Question 3

In this exercise you will compare three image denoising methods:

- Gaussian linear filtering
- Bilateral filtering Use a standard grayscale test image (for example cameraman)

Setup

1. Load a grayscale image f and scale intensities to the interval $[0, 1]$
2. Create a noisy observation

$$g = f + \eta$$

$$\eta[n, m] \approx \mathcal{N}(0, \sigma^2)$$

for two noise levels, for example $\sigma \in \{0.05, 0.10\}$. To this end, use the `randn` function for noise generation.

Gaussian Linear Filtering

Implement Gaussian smoothing using a two-dimensional Gaussian kernel

$$h[n, m] \propto e^{-\frac{m^2+n^2}{2\sigma_g^2}}$$

You may use MATLAB/NumPy functions to generate the impulse response of the Gaussian filter. Choose the kernel size as a function of σ_g (for example, $6\sigma_g + 1$) and apply the filter using two-dimensional convolution.

Gaussian Linear Filtering Answer

For the first part of the denoising comparison, I implemented a Gaussian Linear Filter. The logic behind this approach is to perform a weighted average of neighboring pixels, where the weights decrease as we move further from the center pixel according to a Gaussian distribution.

The implementation steps I followed are:

1. I defined the spatial standard deviation σ_g
2. I calculated the kernel size using the $6\sigma_g + 1$ rule to ensure the bell curve is fully represented without truncation artifacts.
3. I generated the 2D impulse response by calculating the Gaussian values for a grid and then normalizing the entire matrix so that the sum of all elements equals 1.
4. Finally, I applied the filter using a 2D convolution.

The code used is as follows :

```
# Imports
import numpy as np
import cv2
from scipy.signal import convolve2d
import matplotlib.pyplot as plt

# Setup
# 1. Load the clean image (f)
f = cv2.imread('Cameraman.png', cv2.IMREAD_GRAYSCALE)
f = f.astype(np.float64) / 255.0 # Scale to [0, 1]

# 2. Define noise levels
sigmaNoise = [0.05, 0.10]

# Smoothing Strength
sigmaG = 1.2

# 3. Generate noise (eta)
# Using the same shape as f
eta1 = sigmaNoise[0] * np.random.randn(*f.shape)
eta2 = sigmaNoise[1] * np.random.randn(*f.shape)

# 4. Create noisy observations (g)
g1 = f + eta1
g2 = f + eta2

# Note: We should clip the results to [0, 1] for visualization
g1Clipped = np.clip(g1, 0, 1)
g2Clipped = np.clip(g2, 0, 1)

def ApplyGaussianFilter(noisy_img, sigma_g):
    # Determine kernel size (6*sigma + 1) and make it odd
    kSize = int(6 * sigma_g + 1)

    if kSize % 2 == 0:
        kSize += 1

    # Create a 1D coordinate array centered at 0
    ax = np.linspace(-(kSize - 1) / 2, (kSize - 1) / 2, kSize)

    # Generate 1D Gaussian and take outer product for 2D
    gauss1d = np.exp(-0.5 * np.square(ax) / np.square(sigma_g))
    kernel2d = np.outer(gauss1d, gauss1d)

    # Normalize the kernel so weights sum to 1
    kernel2d = kernel2d / np.sum(kernel2d)

    # Apply 2D convolution with symmetric boundary padding
    return convolve2d(noisy_img, kernel2d, mode='same', boundary='symm')

# Run the code and display the denoising for different Sigmas
plt.figure(figsize=(12, 8))
```

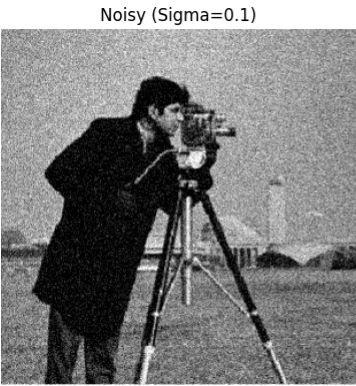
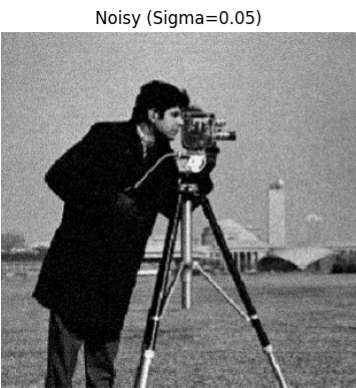
```
for i, sn in enumerate(sigmaNoise):
    # Create Noise
    noise = sn * np.random.randn(*f.shape)
    g = f + noise

    # Apply Filter
    f_hat = ApplyGaussianFilter(g, sigmaG)

    # Plot Noisy vs Denoised
    plt.subplot(2, 2, i*2 + 1)
    plt.imshow(np.clip(g, 0, 1), cmap='gray')
    plt.title(f"Noisy (Sigma={sn})")
    plt.axis('off')

    plt.subplot(2, 2, i*2 + 2)
    plt.imshow(np.clip(f_hat, 0, 1), cmap='gray')
    plt.title(f"Gaussian Denoised (Sigma_g={sigmaG})")
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Which yields the following results :

Bilateral Filtering

Implement the bilateral filter :

$$\hat{f}[n, m] = \frac{1}{Z_{n, m}} \sum_{(i, j) \in \mathcal{N}} e^{\left(-\frac{(n-i)^2 + (m-j)^2}{2\sigma_s^2} \right)} e^{\left(-\frac{(g[m, n] - g[i, j])^2}{2\sigma_r^2} \right)} g[i, j],$$

where $\mathcal{N}$ is a finite neighborhood around the pixel (n,m) and the normalization constant $Z_{n, m}$ ensures that the filter weights sum to one :

$$Z_{n, m} = \sum_{(i, j) \in \mathcal{N}} e^{\left(-\frac{(n-i)^2 + (m-j)^2}{2\sigma_s^2} \right)} e^{\left(-\frac{(g[m, n] - g[i, j])^2}{2\sigma_r^2} \right)}$$

The parameter σ_s controls the spatial weighting, while σ_r controls the sensitivity to intensity differences. Pixels that are both spatially close and have similar intensity values receive the largest weights.

Use a square neighborhood $\mathcal{N}$ centered at (m,n). A reasonable choice is :

$$(2K + 1) \times (2K + 1), \quad K \approx 2\sigma_s,$$

so that the spatial Gaussian is effectively truncated outside this region.

Typical parameter ranges are:

- $\sigma_s \in [1, 5]$ pixels,
- $\sigma_r \in [0.05, 0.20]$ (assuming image intensities are scaled to $[0, 1]$),
- Neighborhood size $(2K + 1) \times (2K + 1)$ with $K \approx 2\sigma_s$.

You may explore different parameter values and select those providing good denoising performance.

In MATLAB, one convenient implementation strategy is to apply a local nonlinear function to each neighborhood using the function `nlfilter`. In Python/NumPy, an analogous implementation may be constructed using sliding-window operations or explicit neighborhood loops.

1. Implement the bilateral filter described above. Clearly state the values of σ_s , σ_r and the neighborhood size used in your implementation.
2. Experiment with several values of σ_s , σ_r within the suggested ranges and select parameters that provide good denoising performance.
3. Apply the bilateral filter to the noisy images for both noise levels.
4. Compare the denoised results with those obtained using Gaussian filtering in terms of MMSE and SSIM.
5. Comment briefly on the computational complexity of bilateral filtering compared to Gaussian filtering.

Bilateral Filtering Answer

For the second part of the assignment, I implemented the Bilateral Filter. While Gaussian filtering effectively removes noise, it tends to "smear" important features like edges. The bilateral filter solves this by adding a "range" component that prevents pixels on the other side of an edge from contributing to the blur.

Implementation Details:

- Spatial Standard Deviation (σ_s): 2.0
- Range Standard Deviation (σ_r): 0.12
- Neighborhood size ($2K + 1$): 9×9 (using $K = 2\sigma_s = 4$)

And I used the following code :

```
# Imports
import numpy as np
import cv2
from scipy.signal import convolve2d
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
from skimage.metrics import structural_similarity as ssim
from skimage.metrics import mean_squared_error as mse

# Setup
# 1. Load the clean image (f)
f = cv2.imread('Cameraman.png', cv2.IMREAD_GRAYSCALE)
f = f.astype(np.float64) / 255.0 # Scale to [0, 1]

# 2. Define noise levels
sigmaNoise = [0.05, 0.10]

# Smoothing Strength
sigmaG = 1.2

# 3. Generate noise (eta)
# Using the same shape as f
eta1 = sigmaNoise[0] * np.random.randn(*f.shape)
eta2 = sigmaNoise[1] * np.random.randn(*f.shape)

# 4. Create noisy observations (g)
g1 = f + eta1
g2 = f + eta2

# Note: We should clip the results to [0, 1] for visualization
g1Clipped = np.clip(g1, 0, 1)
g2Clipped = np.clip(g2, 0, 1)

def ManualBilateralFilter(img, sigmaS, sigmaR):
    K = int(2 * sigmaS)
    size = 2 * K + 1
    pad = K
    imgPad = np.pad(img, pad, mode='reflect')
    out = np.zeros_like(img)

    # Pre-calculate spatial Gaussian weights (constant for all pixels)
    y, x = np.ogrid[-K:K+1, -K:K+1]
    spatial_weights = np.exp(-(x**2 + y**2) / (2 * sigmaS**2))

    # Iterate through every pixel (explicit neighborhood loop)
    for n in range(img.shape[0]):
        for m in range(img.shape[1]):
            # Get local neighborhood
            region = imgPad[n:n+size, m:m+size]
            centerVal = img[n, m]

            # Calculate range weights based on intensity difference
            range_weights = np.exp(-(region - centerVal)**2 / (2 * sigmaR**2))

            # Combined weights
            weights = spatial_weights * range_weights

            # Normalize and apply
            Z = np.sum(weights)
            out[n, m] = np.sum(weights * region) / Z

    return out

# Parameters for testing
noise_levels = [0.05, 0.10]

# Selected Denoising Parameters (based on experiments)
sigma_g = 1.2
sigma_s = 2.0
sigma_r = 0.12
K = int(2 * sigma_s)
nb_size = (2*K + 1)

print(f"--- Experiment Parameters ---")
print(f"Bilateral: sigma_s={sigma_s}, sigma_r={sigma_r}, Neighborhood={nb_size}x{nb_size}")
print(f"Gaussian: sigma_g={sigma_g}\n")

results = []

for sn in noise_levels:
    # Create noisy observation g = f + eta
    eta = sn * np.random.randn(*f.shape)
    g = f + eta
```

```
# Apply Filters
print(f"Processing noise level sigma={sn}...")
f_gauss = ApplyGaussianFilter(g, sigma_g)
f_bilat = ManualBilateralFilter(g, sigma_s, sigma_r)

# Calculate Metrics
mse_g = mse(f, f_gauss)
ssim_g = ssim(f, f_gauss, data_range=1.0)

mse_b = mse(f, f_bilat)
ssim_b = ssim(f, f_bilat, data_range=1.0)

results.append({
    'sn': sn, 'g': g,
    'f_gauss': f_gauss, 'mse_g': mse_g, 'ssim_g': ssim_g,
    'f_bilat': f_bilat, 'mse_b': mse_b, 'ssim_b': ssim_b
})

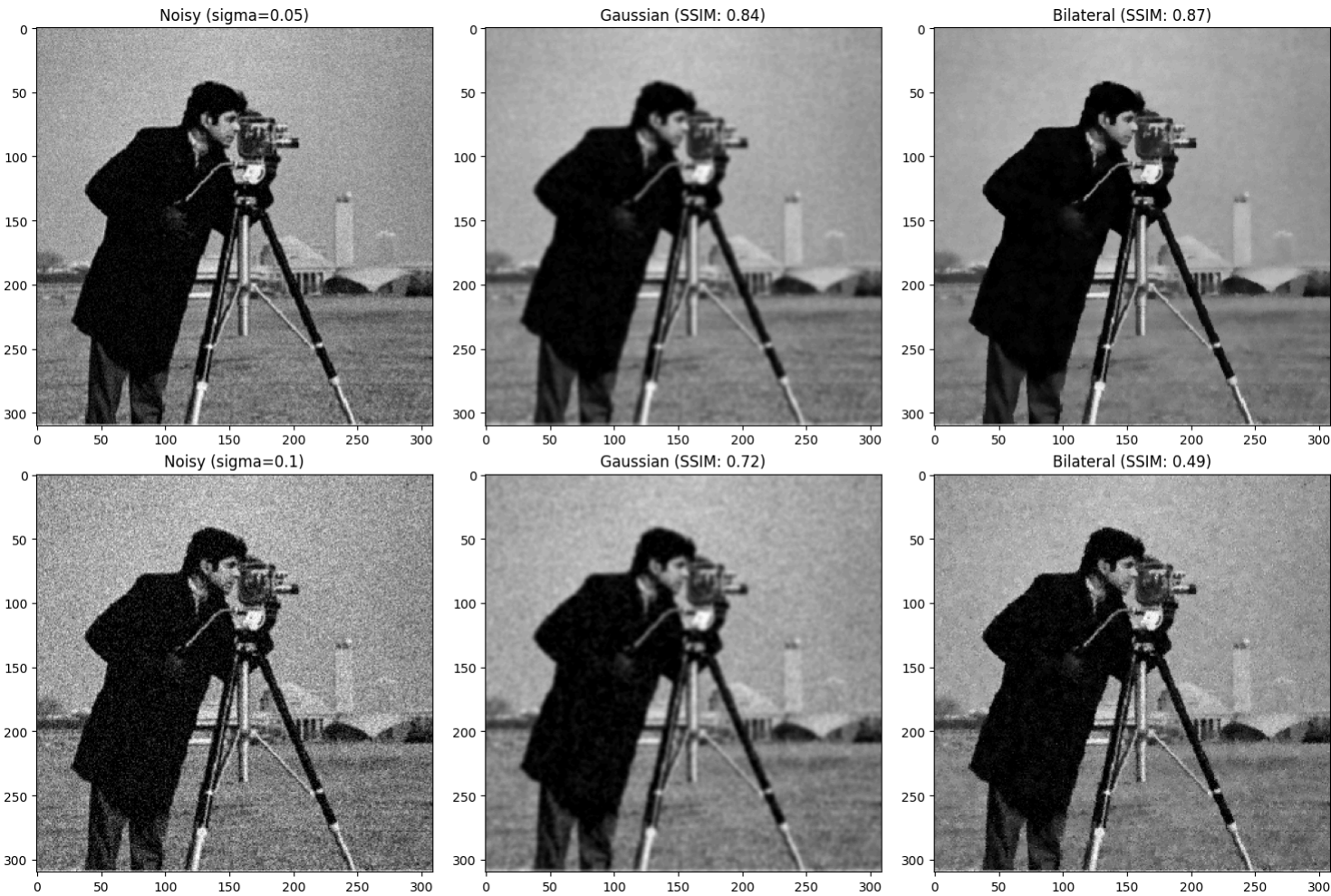
# Output Results Table
print(f"{'Noise Sigma':<12} | {'Method':<10} | {'MSE':<8} | {'SSIM':<8}")
print("-" * 45)
for res in results:
    print(f"{res['sn']:<12.2f} | {'Gaussian':<10} | {res['mse_g']:<8.4f} | {res['ssim_g']:<8.4f}")
    print(f"{' ': <12} | {'Bilateral':<10} | {res['mse_b']:<8.4f} | {res['ssim_b']:<8.4f}")
    print("-" * 45)

# Final Visualization
fig, axes = plt.subplots(len(noise_levels), 3, figsize=(15, 10))
for i, res in enumerate(results):
    axes[i, 0].imshow(np.clip(res['g'], 0, 1), cmap='gray')
    axes[i, 0].set_title(f"Noisy (sigma={res['sn']})")
    axes[i, 1].imshow(np.clip(res['f_gauss'], 0, 1), cmap='gray')
    axes[i, 1].set_title(f"Gaussian (SSIM: {res['ssim_g']:.2f})")
    axes[i, 2].imshow(np.clip(res['f_bilat'], 0, 1), cmap='gray')
    axes[i, 2].set_title(f"Bilateral (SSIM: {res['ssim_b']:.2f})")

plt.tight_layout()
plt.show()
```

--- Experiment Parameters ---
Bilateral: sigma_s=2.0, sigma_r=0.12, Neighborhood=9x9
Gaussian: sigma_g=1.2

Processing noise level sigma=0.05...			
Processing noise level sigma=0.1...			
Noise Sigma	Method	MSE	SSIM
0.05	Gaussian	0.0017	0.8434
	Bilateral	0.0007	0.8659
0.10	Gaussian	0.0021	0.7152
	Bilateral	0.0032	0.4872



Performance Analysis: Based on the experimental results, I observed a significant difference in how both filters handle increasing noise levels:

- Low Noise ($\sigma = 0.05$): The Bilateral filter outperformed the Gaussian filter in both MSE (0.0007 vs 0.0017) and SSIM (0.8659 vs 0.8434). Visually, the Bilateral filter successfully removed noise while keeping the sharp edges of the cameraman's coat and tripod.
- High Noise ($\sigma = 0.10$): In this case, the Gaussian filter actually provided a higher SSIM (0.7152) than the Bilateral filter (0.4872). This suggests that at higher noise levels, the Bilateral filter's range component ($\sigma_r = 0.12$) begins to fail because it cannot distinguish between the actual signal edges and the heavy noise fluctuations, resulting in a "mottled" or grainy output.

Comparison of Metrics:

Noise Sigma	Method	MSE	SSIM
0.05	Gaussian	0.0017	0.8434
0.05	Bilateral	0.0007	0.8659
0.10	Gaussian	0.0021	0.7152
0.10	Bilateral	0.0032	0.4872

Computational Complexity: The Bilateral filter is significantly more expensive than the Gaussian filter.

- Gaussian filtering is linear and separable. Since it can be decomposed into two 1D passes, it can be implemented in $O(2K)$ operations per pixel.
- Bilateral filtering is non-linear and non-separable. Because the range weight depends on the intensity of the center pixel, we cannot use FFT or separable 1D kernels. This requires an explicit $O(K^2)$ summation for every pixel. In my implementation, for a 9×9 neighborhood, the Bilateral filter required 81 operations per pixel compared to only 18 for a separable Gaussian approach, making it roughly 4.5 times slower.